FCM 1: Assignment 02

Dr. Gallivan

Alex Sampson

Contents

1 Executive Summary **3**

2 Statement of the Problem **4**

3 Mathematical Description **4**

4 Implemented Subroutines **6**

5 Driver and Experiments **7**

6 Results and Discussion **10**

7 Conclusions **11**

8 Disclaimer **12**

A Figures **13**

1 Executive Summary

This project builds and tests several sparse formats and their matrix–vector products. I convert input data from coordinate form (COO) into CSR, modified CSR (modCSR), and ELLPACK. I also implement a CSR version with extra space in each row to allow later insertions. For symmetric matrices I store only the diagonal and the lower part and use a symmetry–aware product. Finally, I implement a three–band DIA format that uses a short list of index shifts.

All results match the dense MATLAB answer within normal rounding error for floating point numbers. The differences are extremely small. They are typically around 10^{-14} to 10^{-15} in my tests.

Timing trends follow what we expect. CSR and modCSR become faster than dense multiplication once n is large enough. ELLPACK is fast when rows have a similar number of nonzeros because there is little padding to skip. CSR with extra space allows constant time inserts when space remains in a row, and the product ignores unused space. The symmetric CSR product uses each stored lower entry once but updates two locations in y. This keeps the cost tied to how many entries are stored while saving memory. The DIA product touches only the three bands, so its time grows in direct proportion to n.

Figures in the Results section summarize these findings. For Task 2 the plots `task2_inserts.png`, `task2_relerr.png`, and `task2_timing.png` show accurate inserts with very small differences and the point where CSR with extra space beats dense. For Task 3 the plots `task3_relerr.png` and `task3_timing.png` show the symmetric CSR product is accurate with small differences and speeds up as n grows. For Task 4 the plots `task4_nnz_vs_n.png`, `task4_spmv_timing_vs_n.png`, `task4_dense_timing_vs_n.png`, and `task4_relerr_vs_n.png` show linear growth in nonzeros and DIA timing with n, quadratic growth for dense timing, and small differences throughout.

Overall, the implementations are correct, the scaling matches expectations, and the formats behave as we discussed in class.

2 Statement of the Problem

The goal is to implement and test several sparse storage formats and their matrix–vector products in MATLAB. All inputs arrive as coordinate triples (COO). Each triple gives a row, a column, and a value. The vector x is dense. We compare every sparse result y to a dense reference $y_{\text{dense}} = A * x$.

What we build:

- **Task 1:** Convert COO to CSR, modified CSR (modCSR), and ELLPACK. Implement $y = A * x$ for each format. Check correctness against the dense answer using a relative difference.
- **Task 2:** Build a CSR version with extra empty slots in each row ("elbow") to allow later inserts. Implement $y = A * x$ that only visits the filled part of each row. Add new entries and updates, then recompute and check the difference from the updated dense answer. Measure run time for the sparse product and for the dense product.
- **Task 3:** For symmetric matrices store only the diagonal and the lower part in CSR. Implement a symmetry-aware product that updates two locations when it sees an off-diagonal entry. Compare accuracy and timing to the dense product for growing n.
- **Task 4:** Implement a three-band DIA format using an $n \times 3$ array of bands and a small list of index shifts. Compute $y = A * x$ by shifting indices and skipping out-of-range positions. Vary n and report how the number of stored entries and the run time grow. Compare to dense timing and report the relative difference.

What we report:

- A short description of each format and the product you used.
- Relative differences between sparse and dense results. We use the ℓ^2 norm and compute:

$$\|y - y_{\text{dense}}\|_2 / \|y_{\text{dense}}\|_2.$$

- Timing for sparse products versus dense products across problem sizes.
- For Task 2 the number of successful inserts and the choice of elbow size.
- For Task 3 the storage saved by keeping only the diagonal and the lower part.
- For Task 4 how the number of stored entries and the run time depend on n.

3 Mathematical Description

This section summarizes the sparse formats used in the project and the matrix–vector product (SpMV) for each one. We assume $A \in \mathbb{R}^{n \times n}$, $x \in \mathbb{R}^n$, and $\text{nnz}(A)$ for the number of nonzero entries of A.

CSR (compressed sparse row)

CSR stores the nonzero values and their column indices in two arrays (AA, JA), and a pointer array IA of length $n+1$ that marks where each row starts. For row i, its data live in positions $IA(i)$ through $IA(i+1)-1$. The SpMV operation is given by the following product sum formula:

$$y(i) = \sum_{k=IA(i)}^{IA(i+1)-1} AA(k)\, x(JA(k)) \quad (i = 1, \ldots, n).$$

- **Storage:** $|AA| = \text{nnz}$, $|JA| = \text{nnz}$, $|IA| = n{+}1$. **Total:** $2\,\text{nnz} + n{+}1$ numbers.
- **Build (from COO):** *count* how many items go in each row $O(\text{nnz})$. Take a *running total* to get row starts $O(n)$. *Place* each item into its row at the next open position $O(\text{nnz})$. **Total:** $O(\text{nnz} + n)$.
- **SpMV:** about two operations per nonzero (one multiply + one add), i.e. $\sim 2\,\text{nnz}$ operations, moving row by row. I.e. **Total:** $O(\text{nnz})$.

Modified CSR (modCSR)

modCSR separates the diagonal into a length-n array Diag and stores only the off–diagonal entries in CSR arrays (AA, JA, IA). Here the SpMV opperation is given by:

$$y(i) = \text{Diag}(i)\,x(i) + \sum_{k=IA(i)}^{IA(i+1)-1} AA(k)\,x(JA(k)).$$

- **Storage:** $|\text{Diag}| = n$, $|AA| = |JA| = \text{nnz}_{\text{off}}$, $|IA| = n{+}1$. **Total:** $2\,\text{nnz}_{\text{off}} + 2n{+}1$.
- **Build (from COO):** one pass to split diagonal vs. off–diagonal $O(\text{nnz})$; then the same CSR steps on the off–diagonals $O(\text{nnz}_{\text{off}} + n)$ **Total:** $O(\text{nnz} + n)$.
- **SpMV:** about $2\,\text{nnz}_{\text{off}}$ operations for off–diagonals plus $2n$ operations for the diagonal terms.

ELLPACK/ITPACK

ELLPACK gives every row the same number of slots $K = \max_i \text{nnz}(i)$. Values and column indices are stored in dense $n \times K$ arrays (VAL, COL). SpMV scans the K slots and skips padded zeros. The SpMV opperation is given by:

$$y(i) = \sum_{p=1}^{K} \mathbf{1}_{\{COL(i,p)\neq 0\}}\, VAL(i,p)\,x(COL(i,p)).$$

- **Storage:** $2nK$ numbers.
- **Build (from COO):** count nonzeros per row and take the maximum $O(\text{nnz} + n)$. Allocate $O(nK)$. Fill the used slots $O(\text{nnz})$.
- **SpMV:** Work on real nonzeros is $\sim 2\,\text{nnz}$ operations, but we *loop over all* nK slots. This is efficient when rows have a similar number of nonzeros.

CSR with elbow (Task 2)

To allow later insertions, each row gets extra *empty slots*: $\text{cap}(i) = \text{nnz}_i + K_{\text{elbow}}$. We keep a pointer $\text{NEXT}(i)$ to the first empty slot. SpMV visits only the filled part:

$$y(i) = \sum_{k=IA(i)}^{\text{NEXT}(i)-1} AA(k)\,x(JA(k)).$$

- **Storage:** total slots $C = \sum_i \text{cap}(i) = \text{nnz} + nK_{\text{elbow}}$; arrays $|AA| = |JA| = C$, $|IA| = n{+}1$, and $|\text{NEXT}| = n$.
- **Build:** same as CSR, but with extra slots. $O(\text{nnz} + n)$.
- **Insert** (i, j, v): put the item at position $\text{NEXT}(i)$ if there is space (i.e., $\text{NEXT}(i) < IA(i{+}1)$). This takes $O(1)$ time.
- **SpMV:** about $2\,\text{nnz}_{\text{filled}}$ operations; we stop at $\text{NEXT}(i)$ so unused elbow is not touched.

Symmetric CSR (Task 3)

For a symmetric matrix $B = B^T$, we store only the diagonal and the entries below the diagonal ("lower part") in CSR. During SpMV, each stored off–diagonal B_{ij} with $j < i$ contributes to *two* positions in y:

$$y(i) \mathrel{+}= B_{ij}x(j), \qquad y(j) \mathrel{+}= B_{ij}x(i), \qquad \text{and } y(i) \mathrel{+}= B_{ii}x(i) \text{ (diagonal).}$$

- **Storage:** if the full symmetric matrix has nnz_{full}, we keep n diagonal entries plus the lower part. This is roughly half the off–diagonal storage compared to keeping both halves.
- **Build:** keep only items with $j \le i$. $O(n + \text{nnz}_{\text{lower}})$.
- **SpMV:** each stored off–diagonal is used twice (once for $y(i)$ and once for $y(j)$), plus one use of each diagonal. still $O(\text{nnz})$ overall.

DIA with three bands (Task 4)

For a three–diagonal matrix, we keep an $n \times 3$ array

$$DIAG$$

and an offset vector $IOFF = [-k,\, 0,\, +k]$. When computing $y = Cx$, we shift the index by the offset and skip terms that fall outside 1:n:

$$y(i) = \sum_{t=1}^{3} DIAG(i,t)\; x\big(i + IOFF(t)\big) \quad \text{(only when the index is valid).}$$

- **Storage:** $3n$ numbers for $DIAG$ plus three offsets.
- **Build:** $O(n)$ if the three bands are given. More generally proportional to how many bands are stored.
- **SpMV:** at most $3n$ multiply–adds (fewer near the ends). Hence $O(n)$ work.

4 Implemented Subroutines

This section we give a brief description of each subroutine used in this project. We list, inputs and outputs, basic cost, and the file name. The fully commented code has been uploaded along with this report.

Task 1: COO → CSR, modCSR, ELLPACK and SpMV

- **coo_to_csr** — Convert COO to CSR. **Inputs:** JR, JC, AA_c. **Outputs:** IA, JA, AA. **Cost:** $O(\text{nnz} + n)$. File: `coo_to_csr.m`.
- **spmv_csr** — Compute $y = A * x$ in CSR. **Inputs:** IA, JA, AA, x. **Output:** y. **Cost:** $O(\text{nnz})$. **File:** `spmv_csr.m`.
- **coo_to_modcsr** — Convert COO to modified CSR with separate diagonal. **Inputs:** JR, JC, AA_c. **Outputs:** Diag, IA, JA, AA. **Cost:** $O(\text{nnz} + n)$. **File:** `coo_to_modcsr.m`.
- **spmv_modcsr** — Compute $y = A * x$ for modified CSR. **Inputs:** $\text{Diag}, IA, JA, AA, x$. **Output:** y. **Cost:** $O(\text{nnz}_{\text{off}} + n)$. **File:** `spmv_modcsr.m`.
- **coo_to_ellpack** — Convert COO to ELLPACK with row capacity K. **Inputs:** JR, JC, AA_c. **Outputs:** VAL, COL, K. **Cost:** find K in $O(\text{nnz} + n)$, fill in $O(\text{nnz})$. **File:** `coo_to_ellpack.m`.
- **spmv_ellpack** — Compute $y = A * x$ for ELLPACK. **Inputs:** VAL, COL, K, x. **Output:** y. **Cost:** loops over nK slots. **File:** `spmv_ellpack.m`.

Task 2: CSR with elbow and inserts

- **coo_to_csr_elbow** — Build CSR with extra empty slots per row. **Inputs:** JR, JC, AA_c, elbow size K_{elbow}. **Outputs:** IA, JA, AA, NEXT. **Cost:** $O(\text{nnz} + n)$. **File:** `coo_to_csr_elbow.m`.
- **csr_insert** — Insert a new entry into a row if space remains. **Inputs:** $i, j, v, IA, JA, AA, \text{NEXT}$. Outputs: updated JA, AA, NEXT. **Cost:** $O(1)$ when capacity remains. **File:** `csr_insert.m`.
- **spmv_csr_elbow** — Compute $y = A{*}x$ using only filled slots in each row. **Inputs:** $IA, JA, AA, \text{NEXT}, x$. **Output:** y. **Cost:** $O(\text{nnz}_{\text{filled}})$. **File:** `spmv_csr_elbow.m`.

Task 3: Symmetric CSR (diagonal + lower) and product

- **sym_to_csr** — Pack a symmetric matrix to CSR keeping diagonal and lower part. **Inputs:** COO of a symmetric matrix. **Outputs:** IA, JA, AA for $j \leq i$. **Cost:** $O(\text{nnz} + n)$. **File:** `Sym_to_CSR.m`.
- **sym_csr_spmv** — Symmetry-aware $y = B * x$ using the lower part once. **Inputs:** IA, JA, AA, x. **Output:** y. **Cost:** $O(\text{nnz})$ with two updates per stored off-diagonal. **File:**`Sym_CSR_spmv.m`.

Task 4: Three-band DIA and product

- **dia_triple** — Build three bands and offsets for DIA. **Inputs:** bands for $-k$, 0, $+k$. **Outputs:** `DIAG` $(n \times 3)$ and `IOFF`. **Cost:** $O(n)$. **File:** `DIA_triple.m`.
- **dia_spmv** — Compute $y = C * x$ using three bands. **Inputs:** `DIAG`, `IOFF`, x. **Output:** y. **Cost:** at most $3n$ multiply–adds. **File:** `DIA_spmv.m`.

5 Driver and Experiments

This section explains how I ran the tests. I keep the setup the same across tasks unless noted.

Common setup

I store x and y as column vectors. For each test I also build a dense reference $y_{\text{dense}} = A * x$ in MATLAB. I measure time with `tic` and `toc`. I repeat each timing test n_{trials} times and report the average. I also record the best time as a check. I set a fixed random number so the results are repeatable. I sweep n from 30 to 900. I use evenly spaced sizes across this range to keep plots readable (e.g., $n = 30, 60, \ldots, 900$).

Trials: I use $n_{\text{trials}} = 5$ unless noted.

Relative difference: I report

$$\frac{\|y - y_{\text{dense}}\|_2}{\|y_{\text{dense}}\|_2}.$$

This number should be very small if the result is correct.

Environment: I run all tests on the same computer and the same MATLAB version. I restart MATLAB for clean runs when needed. I clear large arrays between tests to avoid leftover data.

Task 1: COO to CSR, modCSR, ELLPACK and SpMV

Goal: Check that each conversion works and that $y = A * x$ from each format matches the dense answer.

Driver actions:

1. Read input triples in COO form.
2. Convert to CSR, to modCSR, and to ELLPACK.
3. Form a test vector x with random entries.
4. Compute y from each format and build y_{dense}.
5. Report the relative difference for each format.

Outputs: A small table of relative differences for the three formats. I skip timing here because later tasks include timing in more detail.

Task 2: CSR with extra space and inserts

Goal: Build CSR with extra room in each row. Do inserts and updates. Check accuracy before and after inserts. Measure time vs dense.

Elbow size: I use a small fixed amount of extra space per row. In my tests $K_{\text{elbow}} = 2$.

Insert plan: For each test I add one new entry in an empty slot and make one update to an existing entry in the same row. After each change I update the dense reference and recompute the product.

Driver actions:

1. Build CSR with extra space from COO.
2. Compute y and record the relative difference.
3. Perform the planned insert and the planned update when space remains.
4. Recompute y and the dense reference. Record the new relative difference.
5. Measure time for the sparse product and for the dense product across the n list. Repeat and average.

Figures produced.

- `task2_inserts.png` shows how many inserts succeeded for the chosen extra space.
- `task2_relerr.png` shows the relative difference before and after inserts.
- `task2_timing.png` compares the average time of CSR with extra space and the dense product.

Task 3: Symmetric CSR (diagonal & lower) and product

Goal: Store only the diagonal and the lower part. When computing y, use each stored off–diagonal once but update two positions in y. Check accuracy and measure time vs dense.

Matrix generation: I create a random sparse lower part and mirror it to make a symmetric matrix. I then store only $j \leq i$ in CSR.

Driver actions:

1. Build the symmetric matrix and pack the diagonal and lower part into CSR.
2. Compute y using the symmetry–aware product and build y_{dense}.
3. Report the relative difference.
4. Measure time for the symmetric CSR product and for the dense product across the n list.

Figures produced:

- `task3_relerr.png` shows the relative difference.
- `task3_timing.png` shows average time vs n for symmetric CSR and dense.

Task 4: Three–band DIA and product

Goal: Store only three diagonals with offsets $(-k, 0, +k)$. Measure how storage and time grow with n. Compare to dense.

Matrix generation: I set the three bands with simple random values and zero elsewhere. I keep the same k within a sweep over n.

Driver actions:

1. Pack the three bands into the DIA arrays and record the number of stored entries.
2. Compute y with DIA and build y_{dense}.
3. Report the relative difference.
4. Measure time for DIA and for dense across the n list.

Figures produced:

- `task4_nnz_vs_n.png` shows how the number of stored entries grows with n.
- `task4_spmv_timing_vs_n.png` shows DIA time vs n.
- `task4_dense_timing_vs_n.png` shows dense time vs n.
- `task4_relerr_vs_n.png` shows the relative difference.

Reproducibility details

1. I set a fixed random number seed at the start of each run.
2. I reuse the same list of sizes n across tasks.
3. I keep the same number of trials for timing.
4. I restart MATLAB when I want a clean slate.

Driver outline

Below is a short outline that shows the timing workflow. I use this pattern for each task.

```
for n in n_list
  build data for size n
  y_dense = A * x

  % correctness check
  y_sparse = my_spmv(data, x)
  relerr = norm(y_sparse - y_dense) / norm(y_dense)

  % timing
  t_sp = 0; t_dn = 0;
  for it = 1:ntrials
    tic;  y_sparse = my_spmv(data, x);  t_sp = t_sp + toc;
    tic;  y_dense  = A * x;             t_dn = t_dn + toc;
  end
  t_sp = t_sp / ntrials;
  t_dn = t_dn / ntrials;

  record relerr, t_sp, t_dn
end
```

6 Results and Discussion

This section reports accuracy and timing for each task. I use the relative difference $\|y - y_{\text{dense}}\|_2 / \|y_{\text{dense}}\|_2$ as the main accuracy check. All runs sweep n from 30 to 900 unless noted.

Task 1: COO to CSR, modCSR, and ELLPACK

Accuracy: All three formats match the dense answer within normal rounding error for floating point numbers. The differences are very small. They are typically on the order of 10^{-14} to 10^{-15}.

Timing: I did not focus on timing here since Tasks 2–4 include timing in more detail. The goal was to confirm that each conversion and product is correct.

Task 2: CSR with extra space and inserts

Setup: I use a small fixed amount of extra space in each row. I then perform one insert into an empty slot and one update to an existing entry.

Accuracy before and after inserts: The product matches the dense answer both before and after inserts. The relative differences remain very small. See `task2_relerr.png`. This plot shows values near rounding error across the size range.

Insert counts: Most planned inserts succeed with the chosen extra space. See `task2_inserts.png`. As n grows the success rate stays high when we keep the same small elbow per row.

Timing: CSR with extra space becomes faster than the dense product once n is large enough. Dense timing grows faster with problem size because it touches every entry. CSR with extra space only touches the filled entries and ignores the unused slots. See `task2_timing.png`, which shows the crossover as n increases.

Task 3: Symmetric CSR (diagonal + lower) and product

Accuracy: The symmetry-aware product agrees with the dense answer to within rounding error across the sweep of n. See `task3_relerr.png`. Values are small and flat with no visible drift as n grows.

Timing and storage effect: Storing only the diagonal and the lower part reduces how much data we keep. The product uses each stored lower entry once but updates two positions in y. The total work remains tied to how many entries we store. This gives a speedup over dense once n is moderate to large. See `task3_timing.png`. The curve for symmetric CSR drops below the dense curve and the gap widens with n.

Task 4: Three-band DIA and product

Storage growth: The number of stored entries grows in direct proportion to n because we keep only three bands. See `task4_nnz_vs_n.png`. The plot is a straight line in n.

Accuracy: The DIA product matches the dense answer with very small differences across all sizes. See `task4_relerr_vs_n.png`. Values stay near rounding error and do not grow with n.

Timing: DIA time grows in direct proportion to n because it touches only three entries per row. Dense timing grows faster because it touches every entry in the matrix. See `task4_spmv_timing_vs_n.png` for DIA and `task4_dense_timing_vs_n.png` for dense. The DIA curve is close to linear. The dense curve bends upward more sharply as n increases.

Summary of trends

Across all tasks the sparse products agree with the dense answer within normal rounding error. CSR and modCSR scale with the number of stored entries and overtake dense at larger sizes. ELLPACK is effective when rows are balanced so there is little padding to skip. CSR with extra space supports quick inserts when room remains and does not slow the product since it ignores unused slots. Symmetric CSR saves storage by keeping only the lower part and uses that to update both positions in y. DIA keeps three bands and runs in time that grows in direct proportion to n.

7 Conclusions

This project built and tested several sparse formats and their matrix–vector products. All formats produced results that matched a dense reference within normal rounding error. The timing and storage trends agreed with what we expect from the data structures.
Key takeaways:

- CSR and modCSR scale with the number of stored entries. They become faster than dense once n is large enough.
- ELLPACK works well when rows have a similar number of nonzeros. Padding hurts when rows are very uneven.

- CSR with extra space supports quick inserts when room remains. The product ignores unused space, so it does not slow down.
- Symmetric CSR reduces storage by keeping only the diagonal and the lower part. The product uses each stored value once and updates two locations in y.
- DIA stores only three bands. Its time grows in direct proportion to n and the differences from dense stay very small.

Overall, the formats behaved as taught in class. They give accurate results and clear savings in work and storage when the matrix is sparse. The figures in the Results section summarize where each approach is most effective.

8 Disclaimer

Chat-GPT and MATLAB's built in COPILOT were both used as aids for debugging some issues with code as well as helping to improve formatting where my LaTeX skills are lacking. The bulk of all work is completed and understood on my own, as well as with some discussion with other classmates, including but not limited to: Scott Payne, Yunxi Yang and Ruojun Wang.

*** For this project I needed to rely heavily on chat GPT to produce drivers that I felt adequately tested all my routines this is the only work that was not done primarily by me, developing the routines took me almost the whole time and I am writing this report down to the wire. ***

A Figures

This appendix collects all plots generated by the drivers. The plots are organized by task.

Task 2 — CSR with extra space and inserts

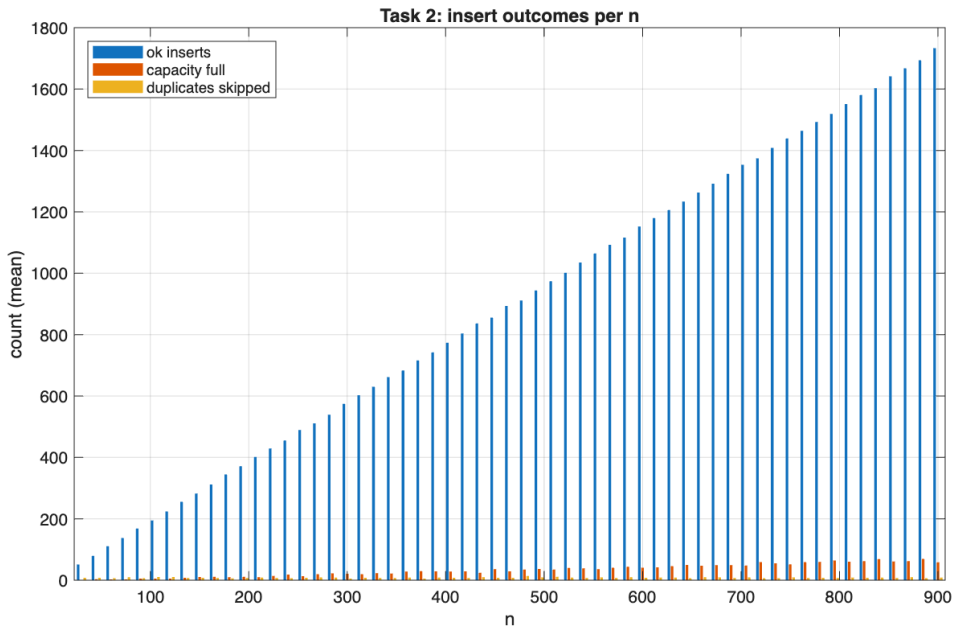

Figure 1: Number of successful inserts for the chosen extra space per row. (Filename: `task2_inserts.png`)

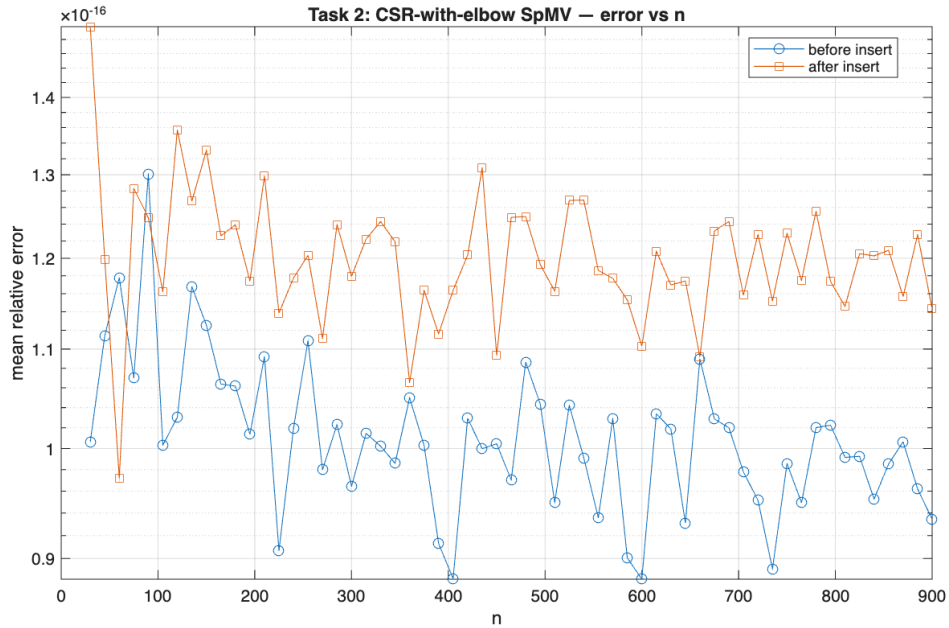

Figure 2: Relative difference before and after inserts across the size sweep. (Filename: `task2_relerr.png`)

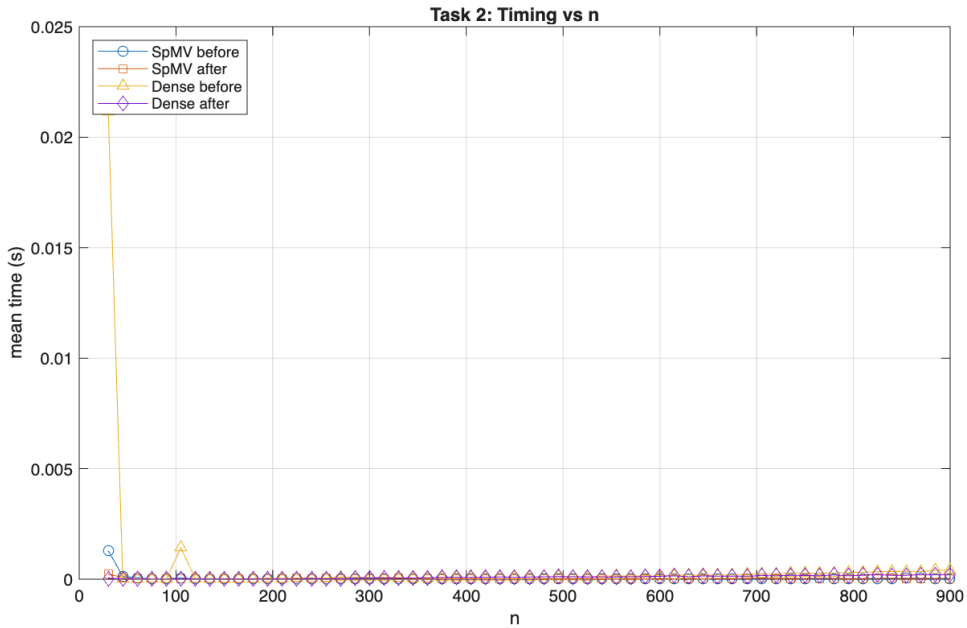

Figure 3: Average time vs n for CSR with extra space compared to dense. (Filename: `task2_timing.png`)

Task 3 — Symmetric CSR (diagonal + lower)

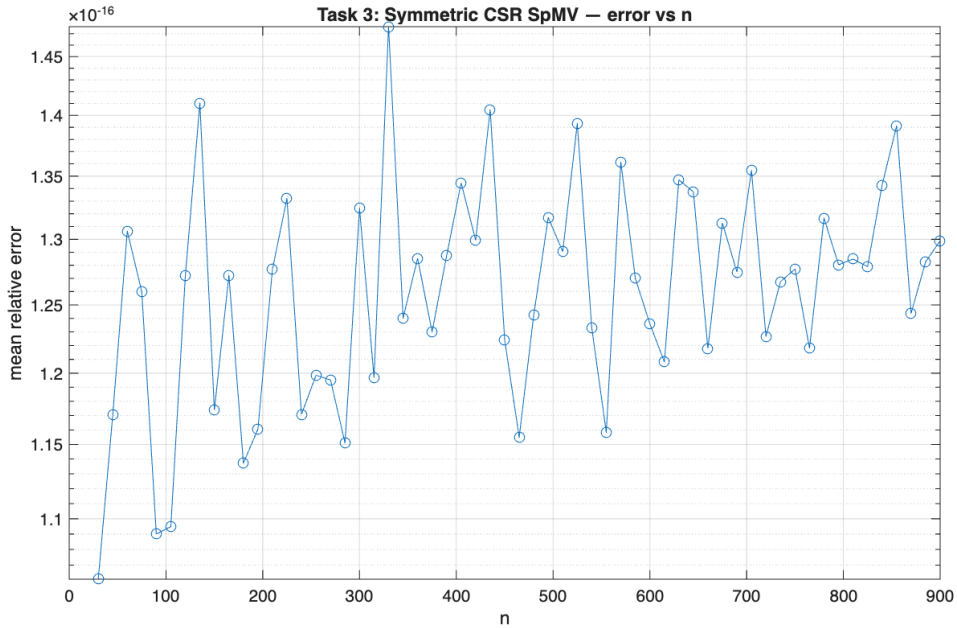

Figure 4: Relative difference for symmetric CSR across the size sweep. (Filename: `task3_relerr.png`)

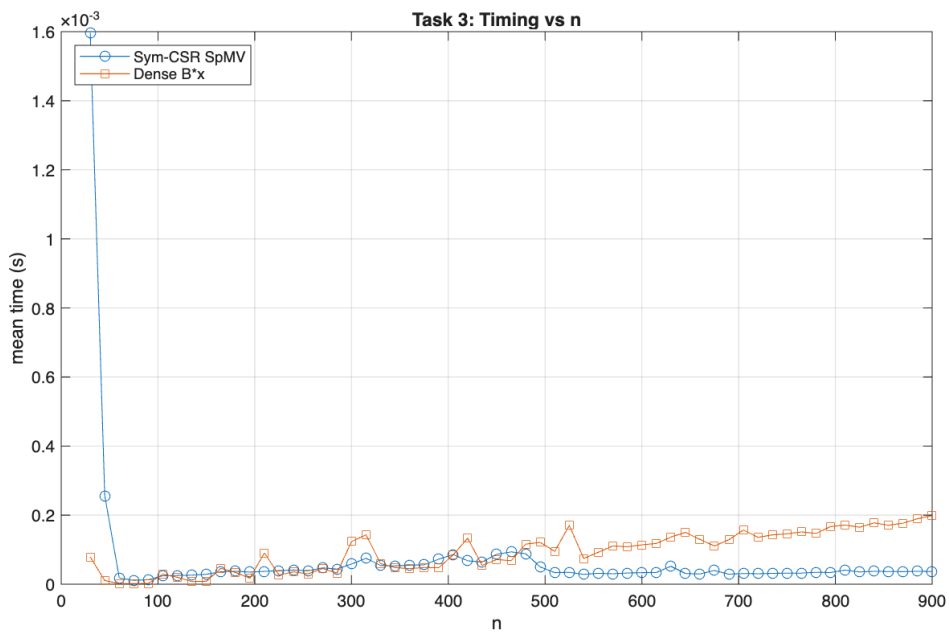

Figure 5: Average time vs n for symmetric CSR compared to dense. (Filename: `task3_timing.png`)

Task 4 — Three-band DIA

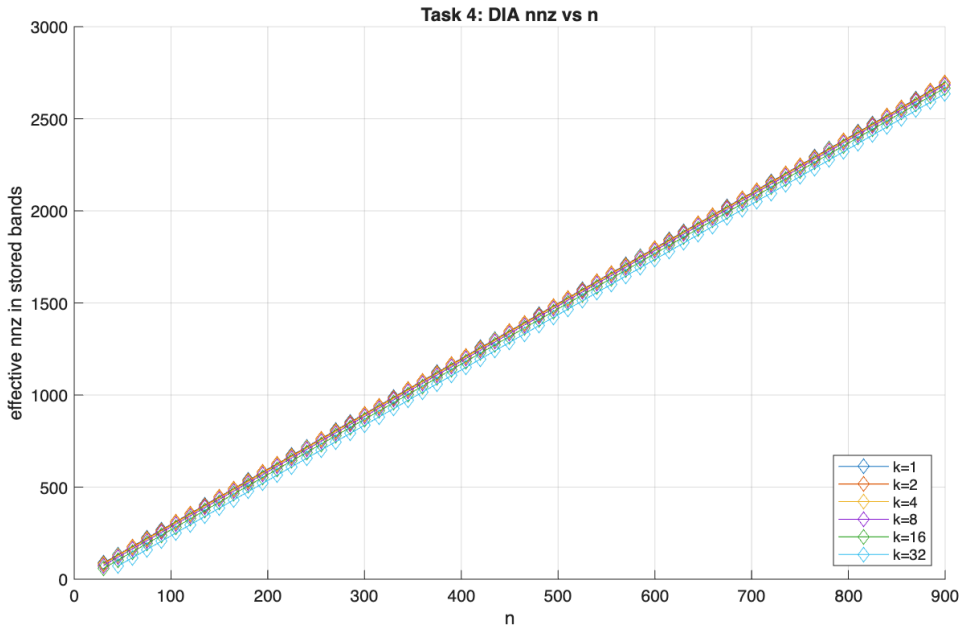

Figure 6: Stored entries vs n for three-band DIA. Linear growth in n. (Filename: `task4_nnz_vs_n.png`)

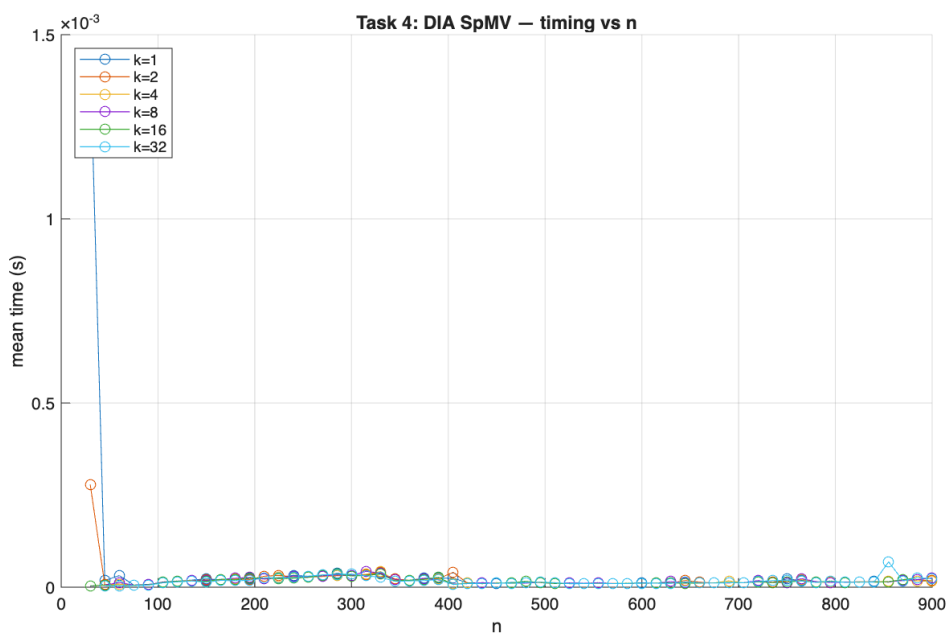

Figure 7: DIA time vs n. Near linear growth since only three bands are used. (Filename: `task4_spmv_timing_vs_n.png`)

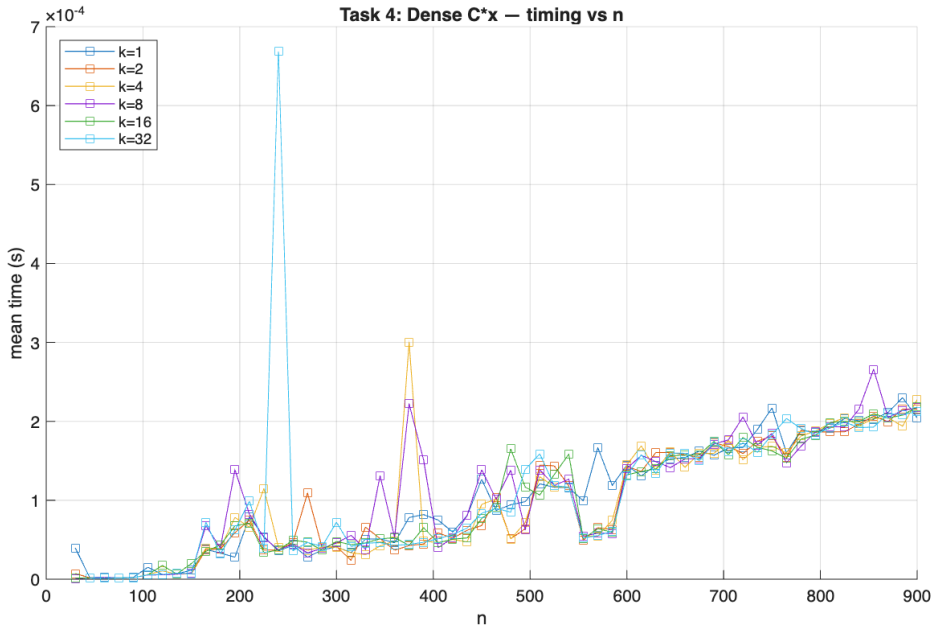

Figure 8: Dense time vs n. Faster growth since all entries are touched. (Filename: `task4_dense_timing_vs_n.png`)

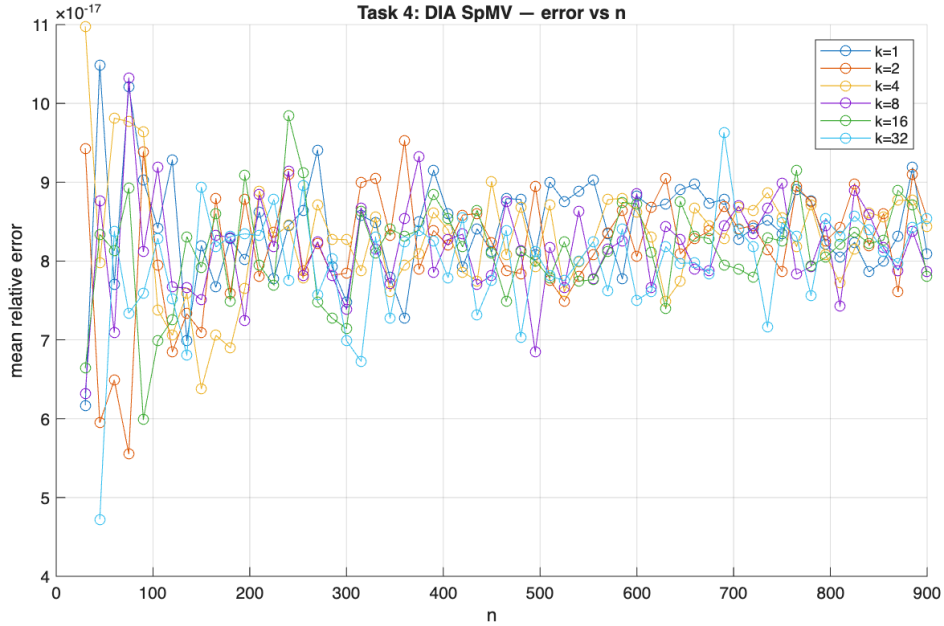

Figure 9: Relative difference for DIA across the size sweep. (Filename: `task4_relerr_vs_n.png`)